LAPORAN PRAKTIKUM
KECERDASAN BUATAN
Text Classification



Fani Namesia R. Sianturi
42324045
D3 Teknologi Informasi

INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
TAHUN AJARAN
2026/2027

```
Generate  Code  Markdown

import pandas as pd
df = pd.read_csv('Text_classification.csv')
print(df.head())

[2]  ✓ 0.0s                                                    Python

  Category                                            Message
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
```

Penjelasan:

- Import pandas. import pandas as pd memuat library pandas yang dipakai untuk mengolah data berbentuk tabel. Alias pd adalah singkatan standar supaya penulisan kode lebih ringkas.
- Baca file CSV. pd.read_csv('Text_classification.csv') membaca file Text_classification.csv dan menyimpannya ke variabel df dalam bentuk DataFrame (tabel dengan baris dan kolom). File harus berada di folder yang sama dengan notebook, atau gunakan path lengkap jika berada di folder lain.
- Tampilkan 5 baris pertama. df.head() menampilkan lima baris teratas dataset, dan print(...) mencetaknya sebagai output.

```
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
# Import library untuk preprocessing
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
# Inisialisasi lemmatizer
lemmatizer = WordNetLemmatizer()
# Fungsi preprocessing teks
def clean_text(text):
    text = text.lower() # Ubah menjadi lowercase
    text = re.sub(r'\d+', '', text) # Hapus angka
    text = text.translate(str.maketrans('', '', string.punctuation)) # Hapus tanda baca
    words = word_tokenize(text) # Tokenisasi
    words = [word for word in words if word not in stopwords.words('english')] # Hapus stopwords
    words = [lemmatizer.lemmatize(word) for word in words] # Lemmatization
    return ' '.join(words)
# Terapkan preprocessing ke teks
df['clean_text'] = df['Message'].apply(clean_text)
# Cek hasil preprocessing
print("\nHasil Preprocessing:\n", df[['Message', 'clean_text']].head())

[8]  ✓ 4m 23.6s                                                Python

[nltk_data] Downloading package punkt_tab to C:\Users\FANI
[nltk_data]     SIANTURI\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package punkt to C:\Users\FANI
[nltk_data]     SIANTURI\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to C:\Users\FANI
[nltk_data]     SIANTURI\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to C:\Users\FANI
[nltk_data]     SIANTURI\AppData\Roaming\nltk_data...

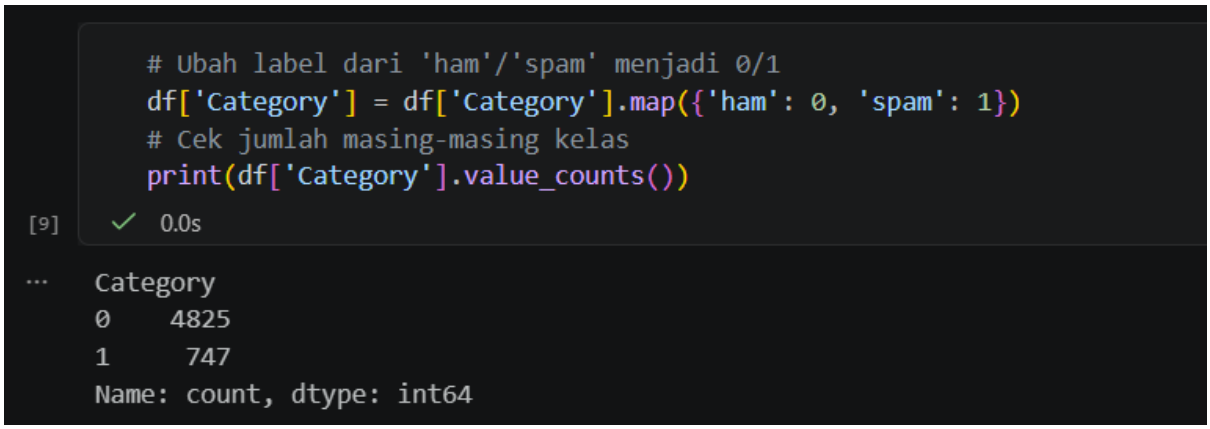
Hasil Preprocessing:
                                             Message  \
0  Go until jurong point, crazy.. Available only ...
1                      Ok lar... Joking wif u oni...
2  Free entry in 2 a wkly comp to win FA Cup fina...
3  U dun say so early hor... U c already then say...
4  Nah I don't think he goes to usf, he lives aro...

                                          clean_text
0  go jurong point crazy available bugis n great ...
1                              ok lar joking wif u oni
2  free entry wkly comp win fa cup final tkts st ...
3                u dun say early hor u c already say
4           nah dont think go usf life around though
```

Penjelasan:
Cell ini membersihkan teks pesan agar seragam sebelum diubah menjadi angka untuk model. Pertama, library NLTK diimpor dan resource yang dibutuhkan diunduh, yaitu punkt untuk tokenisasi, stopwords untuk daftar kata umum bahasa Inggris, dan wordnet untuk lemmatisasi.

Inti cell ini adalah fungsi clean_text, yang memproses setiap pesan dalam enam tahap: mengubah semua huruf menjadi huruf kecil, menghapus angka, menghapus tanda baca, memecah teks menjadi kata (tokenisasi), membuang kata umum (stopwords) seperti “the” dan “is”, lalu mengembalikan setiap kata ke bentuk dasarnya (lemmatisasi). Kata-kata yang tersisa digabungkan kembali menjadi satu kalimat.

Fungsi tersebut diterapkan ke kolom Message dan hasilnya disimpan di kolom baru clean_text. Terakhir, lima baris pertama ditampilkan untuk membandingkan teks asli dan teks bersih. Sebagai contoh, “Congratulations! You won a free trip to Bali.” menjadi “congratulation free trip bali”. Dengan begitu, kata penciri spam seperti “free” lebih menonjol dan model dapat membedakan spam dan ham dengan lebih baik.



Penjelasan:

Cell ini mengubah label teks pada kolom Category menjadi angka, karena model machine learning hanya dapat memproses data numerik. Dengan map({'ham': 0, 'spam': 1}), pesan normal (ham) diberi nilai 0 dan pesan spam diberi nilai 1, lalu hasilnya menggantikan isi kolom Category.

Setelah itu, value_counts() menghitung jumlah data pada masing-masing kelas dan menampilkannya. Hasil ini berguna untuk melihat keseimbangan data, karena jumlah ham biasanya jauh lebih banyak daripada spam, sehingga akurasi saja tidak cukup untuk menilai model.

Cell ini sebaiknya hanya dijalankan sekali. Jika dijalankan dua kali, kolom yang sudah berisi angka akan berubah menjadi NaN, sehingga data perlu dimuat ulang terlebih dahulu.

```
Click to add a breakpoint selection import train_test_split
# Pisahkan fitur dan label
X = df['clean_text']
y = df['Category']
# Split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
[10] ✓ 0.5s
```

Penjelasan:
Cell ini membagi dataset menjadi data latih (training) dan data uji (testing). Pertama, fitur X diambil dari kolom clean_text (teks yang sudah dibersihkan), sedangkan label y diambil dari kolom Category (0 untuk ham, 1 untuk spam).

Selanjutnya, train_test_split membagi data dengan proporsi 80% untuk training dan 20% untuk testing (test_size=0.2). Parameter random_state=42 membuat pembagian selalu sama setiap kali dijalankan sehingga hasilnya dapat direproduksi, dan stratify=y menjaga proporsi ham dan spam di data latih maupun data uji tetap sama dengan dataset asli. Hal ini penting karena data spam jumlahnya jauh lebih sedikit.

Hasilnya adalah empat variabel: X_train dan y_train untuk melatih model, serta X_test dan y_test untuk menguji performa model pada data yang belum pernah dilihat sebelumnya.

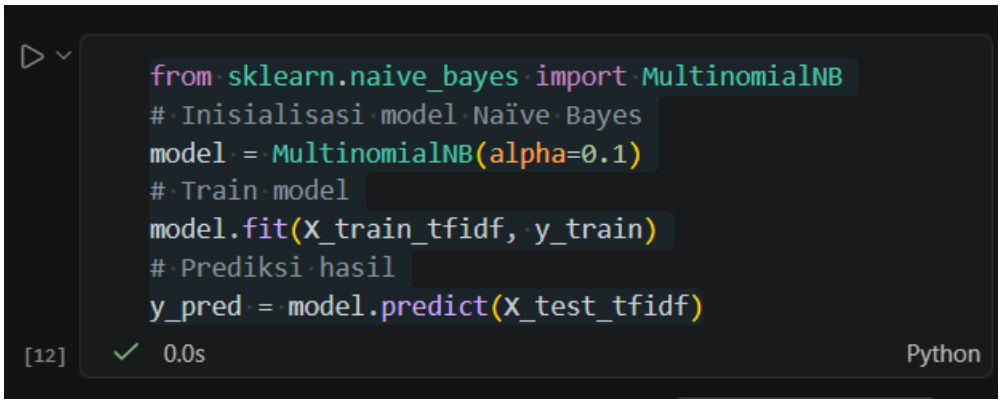
```
from sklearn.feature_extraction.text import TfidfVectorizer
# Konversi teks menjadi fitur numerik menggunakan TF-IDF
vectorizer = TfidfVectorizer(ngram_range=(1,2), max_features=5000) # Menggunakan bigram
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("\nTF-IDF Shape (Train):", X_train_tfidf.shape)
[11] ✓ 0.2s
...
TF-IDF Shape (Train): (4457, 5000)
```

Penjelasan:
Cell ini mengubah teks yang sudah dibersihkan menjadi angka (fitur numerik), karena model machine learning tidak bisa memproses teks secara langsung. Metode yang dipakai adalah TF-IDF, yang memberi bobot pada setiap kata berdasarkan seberapa sering kata itu muncul di sebuah pesan (TF) dan seberapa jarang kata itu muncul di seluruh pesan (IDF). Kata yang sering muncul di satu pesan tetapi jarang di pesan lain akan mendapat bobot tinggi, sehingga kata penciri seperti "free" atau "prize" lebih diperhatikan model.

Parameter `ngram_range=(1,2)` membuat vectorizer memakai unigram (satu kata) sekaligus bigram (dua kata berurutan, misalnya "free trip"), sehingga sebagian konteks antar kata ikut tertangkap. Parameter `max_features=5000` membatasi fitur pada 5.000 kata atau frasa dengan bobot terpenting agar model lebih ringan.

`fit_transform` pada data latih mempelajari kosakata dan bobot IDF dari `X_train`, lalu mengubahnya menjadi matriks angka. Sementara itu, data uji hanya diproses dengan `transform` memakai kosakata dari data latih, supaya tidak terjadi kebocoran data (data

leakage). Terakhir, `X_train_tfidf.shape` mencetak ukuran matriks, berupa jumlah pesan latih dikali jumlah fitur (maksimal 5.000).



Penjelasan:
Cell ini melatih model klasifikasi spam menggunakan algoritma Multinomial Naïve Bayes. Algoritma ini cocok untuk data teks karena bekerja dengan menghitung probabilitas sebuah pesan termasuk spam atau ham berdasarkan bobot kata-kata di dalamnya, dan prosesnya cepat meski datanya berdimensi tinggi.

Model dibuat dengan MultinomialNB(alpha=0.1). Parameter alpha adalah smoothing yang mencegah probabilitas bernilai nol untuk kata yang tidak pernah muncul pada salah satu kelas saat pelatihan. Nilai 0,1 lebih kecil dari default (1,0), sehingga model lebih sensitif terhadap kata-kata yang benar-benar khas.

Selanjutnya, model.fit(X_train_tfidf, y_train) melatih model dengan fitur TF-IDF data latih beserta labelnya. Setelah itu, model.predict(X_test_tfidf) dipakai untuk memprediksi label data uji, dan hasilnya disimpan di y_pred (0 untuk ham, 1 untuk spam). Variabel y_pred inilah yang nanti dibandingkan dengan y_test pada tahap evaluasi, seperti confusion matrix.

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Akurasi
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

[13] ✓ 0.0s

```
Accuracy: 0.979372197309417

Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       966
           1       0.98      0.86      0.92       149

    accuracy                           0.98      1115
   macro avg       0.98      0.93      0.95      1115
weighted avg       0.98      0.98      0.98      1115
```

Penjelasan:

Cell ini mengukur seberapa baik model memprediksi data uji dengan membandingkan label asli (y_test) dan hasil prediksi (y_pred). Fungsi accuracy_score menghitung akurasi, yaitu persentase pesan yang diprediksi dengan benar dari seluruh data uji. Nilainya berkisar 0 sampai 1, dan semakin mendekati 1 berarti semakin baik.

Karena data spam jumlahnya jauh lebih sedikit daripada ham, akurasi saja bisa menyesatkan, sehingga classification_report juga ditampilkan. Laporan ini berisi empat metrik untuk tiap kelas. Precision menunjukkan dari semua pesan yang diprediksi spam, berapa yang benar-benar spam. Recall menunjukkan dari semua pesan spam yang sebenarnya, berapa yang berhasil terdeteksi. F1-score adalah rata-rata harmonik precision dan recall, dan support adalah jumlah data asli pada tiap kelas. Model yang baik memiliki precision dan recall tinggi, terutama pada kelas spam (1).

Perhatikan bahwa confusion_matrix ikut diimpor di cell ini tetapi belum dipakai, karena visualisasinya dibuat pada cell terpisah setelahnya.

```python
# Confusion Matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix

# Hitung confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Definisi elemen dalam confusion matrix
TN, FP, FN, TP = conf_matrix.ravel()

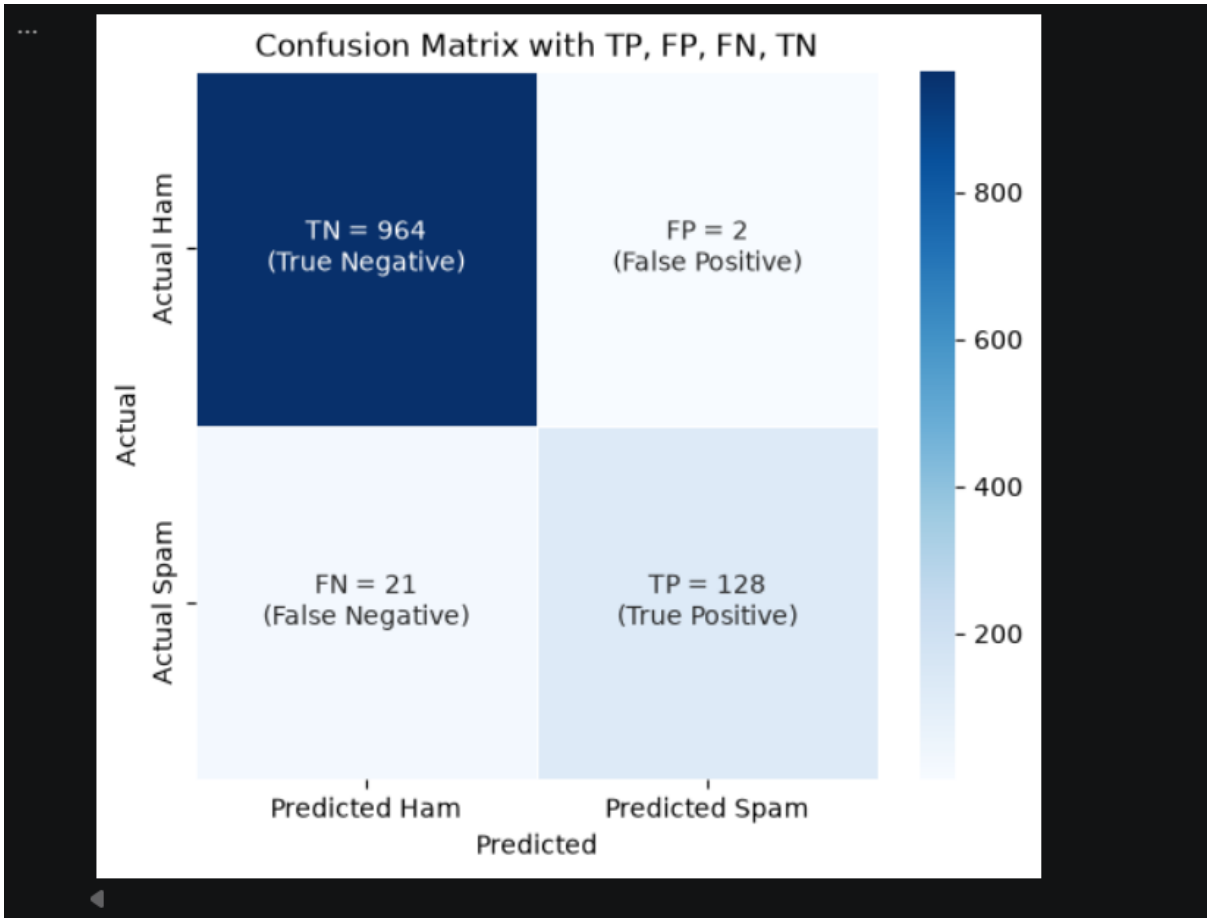
# Buat dataframe confusion matrix
df_conf_matrix = pd.DataFrame(conf_matrix, index=['Actual Ham', 'Actual Spam'], columns=['Predicted Ham', 'Predicted Spam'])

# Label untuk TP, FP, FN, TN
labels = np.array([["TN = {}\n(True Negative)".format(TN), "FP = {}\n(False Positive)".format(FP)],
                   ["FN = {}\n(False Negative)".format(FN), "TP = {}\n(True Positive)".format(TP)]])

# Plot heatmap dengan keterangan
plt.figure(figsize=(6,5))
ax = sns.heatmap(df_conf_matrix, annot=labels, fmt='', cmap='Blues', linewidths=0.5, cbar=True)

# Tambahkan label
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix with TP, FP, FN, TN')
plt.show()
```

[17] ✓ 0.3s Python



Penjelasan:

Cell ini memvisualisasikan hasil prediksi model dalam bentuk confusion matrix, yaitu tabel 2x2 yang membandingkan label asli (y_test) dengan hasil prediksi (y_pred). Setelah matriks dihitung dengan confusion_matrix, fungsi ravel() meratakannya menjadi empat nilai: TN (ham yang diprediksi ham), FP (ham yang salah diprediksi spam), FN (spam yang salah diprediksi ham), dan TP (spam yang diprediksi spam).

Matriks tersebut kemudian dimasukkan ke DataFrame dengan nama baris dan kolom (Actual/Predicted Ham dan Spam) agar mudah dibaca. Array labels dibuat untuk menampilkan keterangan beserta jumlahnya di setiap kotak, misalnya "TN = 950 (True Negative)". Selanjutnya, sns.heatmap menggambar matriks sebagai peta warna biru, di mana warna yang lebih gelap menunjukkan jumlah yang lebih besar, lalu judul dan label sumbu ditambahkan dan grafik ditampilkan dengan plt.show().

Cara membacanya: diagonal utama (TN dan TP) adalah prediksi yang benar, sedangkan FP dan FN adalah kesalahan model. Model yang baik memiliki angka besar pada diagonal dan angka kecil pada FP dan FN.

```
# Contoh teks baru
new_texts = ["Congratulations! You won a free trip to Bali.",
 "Hey, are you coming to the party tonight?"]
# Preprocess teks baru
new_texts_clean = [clean_text(text) for text in new_texts]
# Transformasi dengan TF-IDF
new_texts_tfidf = vectorizer.transform(new_texts_clean)
# Prediksi
predictions = model.predict(new_texts_tfidf)
# Output hasil prediksi
for text, label in zip(new_texts, predictions):
 print(f"\nMessage: {text} \nPredicted: {'Spam' if label == 1 else 'Ham'}")
```

[18] ✓ 0.0s

```
Message: Congratulations! You won a free trip to Bali.
Predicted: Spam

Message: Hey, are you coming to the party tonight?
Predicted: Ham
```

Penjelasan:

Cell ini menguji model yang sudah dilatih pada pesan baru yang belum pernah dilihat sebelumnya. Dua contoh pesan disimpan dalam list new_texts: satu pesan yang terlihat seperti spam ("Congratulations! You won a free trip to Bali.") dan satu pesan biasa ("Hey, are you coming to the party tonight?").

Sebelum diprediksi, pesan baru harus diproses dengan cara yang sama seperti data latih. Karena itu, setiap pesan dibersihkan dengan fungsi clean_text, lalu diubah menjadi fitur numerik dengan vectorizer.transform. Perhatikan bahwa yang dipakai adalah transform, bukan fit_transform, supaya kosakata dan bobot yang dipelajari dari data latih tetap dipakai. Setelah itu, model.predict menghasilkan label untuk tiap pesan (0 untuk ham, 1 untuk spam).

Terakhir, perulangan for dengan zip mencetak setiap pesan beserta hasil prediksinya, dengan mengubah angka 1 menjadi "Spam" dan angka 0 menjadi "Ham". Jika model bekerja dengan

baik, pesan pertama akan diprediksi Spam dan pesan kedua Ham, seperti yang terlihat pada output di screenshot kamu sebelumnya.